Assignment-1

Submitted BY:Fizza Shameen

Reg. No.:23-NTU-CS-1157

Submitted To: Sir Nasir

Submission Date:26-1-2025

Section A:

Task1:

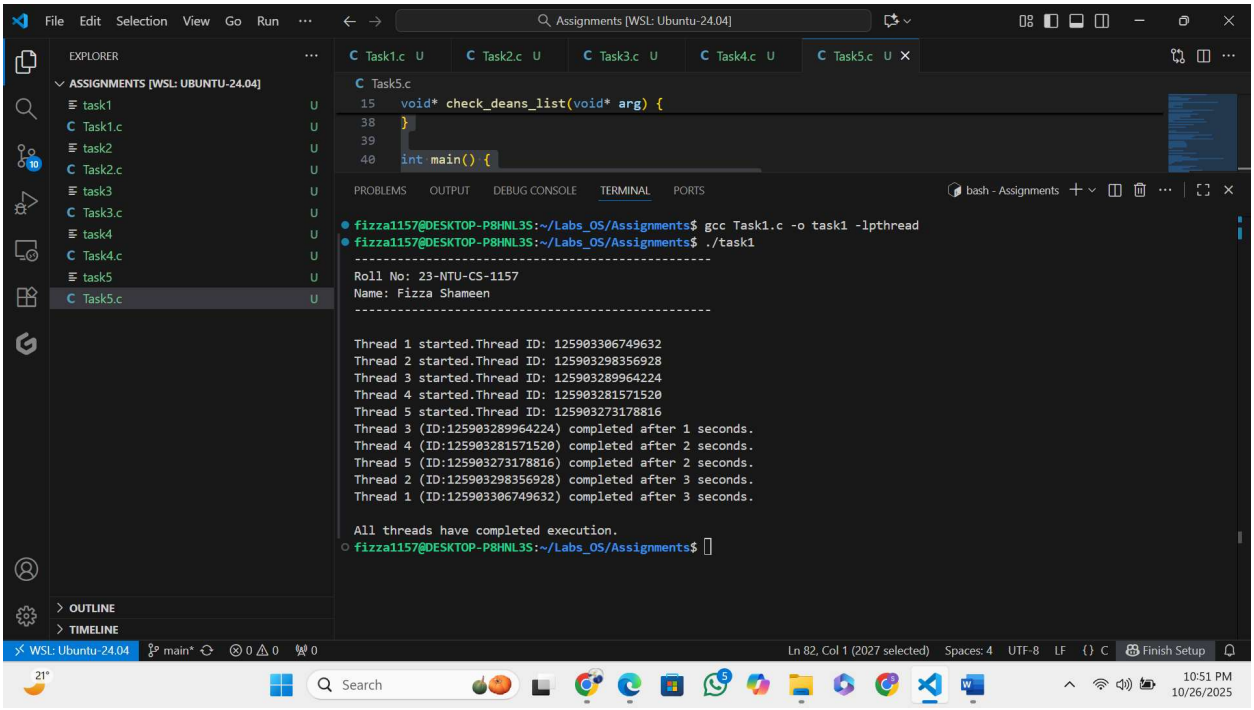
Solution:

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>

//Function executed by each thread
void* thread_func(void* arg){
   int thread_num = *(int*)arg;    //Get thread number(1-5)
   pthread_t tid = pthread_self(); //Get thread id
   printf("Thread %d started.Thread ID: %lu\n",thread_num,tid);

   //Sleep for a random time between 1-3 seconds
   int sleep_time = rand() % 3 + 1;
   sleep(sleep_time);
```

```
    printf("Thread %d (ID:%lu) completed after %d seconds.\n",thread_num,tid,sleep_time);
    pthread_exit(NULL);
}
int main(){
    pthread_t threads[5];
    int thread_nums[5];
    srand(time(NULL));   //Random number generator

    //Create 5 threads
    for(int i=0;i < 5;i++){
        thread_nums[i] = i + 1;
        if(pthread_create(&threads[i],NULL,thread_func,&thread_nums[i])!= 0){
            perror("Failed to create thread");
            return 1;
        }
    }
    for(int i=0;i < 5; i++){
        pthread_join(threads[i], NULL);
    }
    printf("\nAll threads have completed execution.\n");
    return 0;
}
```

Output:



Task2:

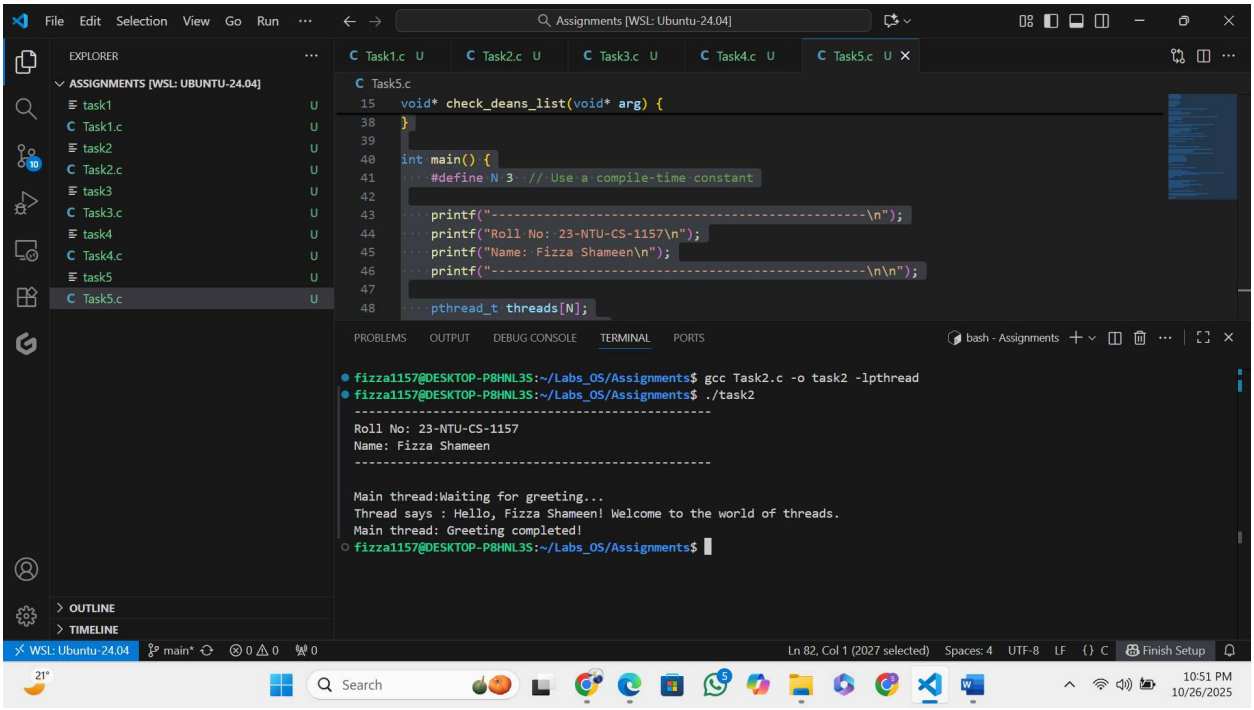
Solution:

Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>


//Function executed by each thread

void *greet(void* arg){

  char* name = (char*) arg; //Get name passed as argument

  printf("Thread says : Hello, %s! Welcome to the world of threads.\n",name);

  pthread_exit(NULL);
```

```
}
int main(){
   pthread_t thread;
   char* name = "Fizza Shameen"; //Name passed to thread
   printf("-------------------------------------------------\n");
   printf("Roll No: 23-NTU-CS-1157\n");
   printf("Name: Fizza Shameen\n");
   printf("-------------------------------------------------\n\n");

   // Create thread
   if(pthread_create(&thread,NULL,greet,name)!=0){
      perror("Failed to create thread.");
      return 1;
   }
   printf("Main thread:Waiting for greeting...\n");

   //Wait for the greeting thread to finish
   pthread_join(thread,NULL);
   printf("Main thread: Greeting completed!\n");
   return 0;
}
```

Output:



Task3:

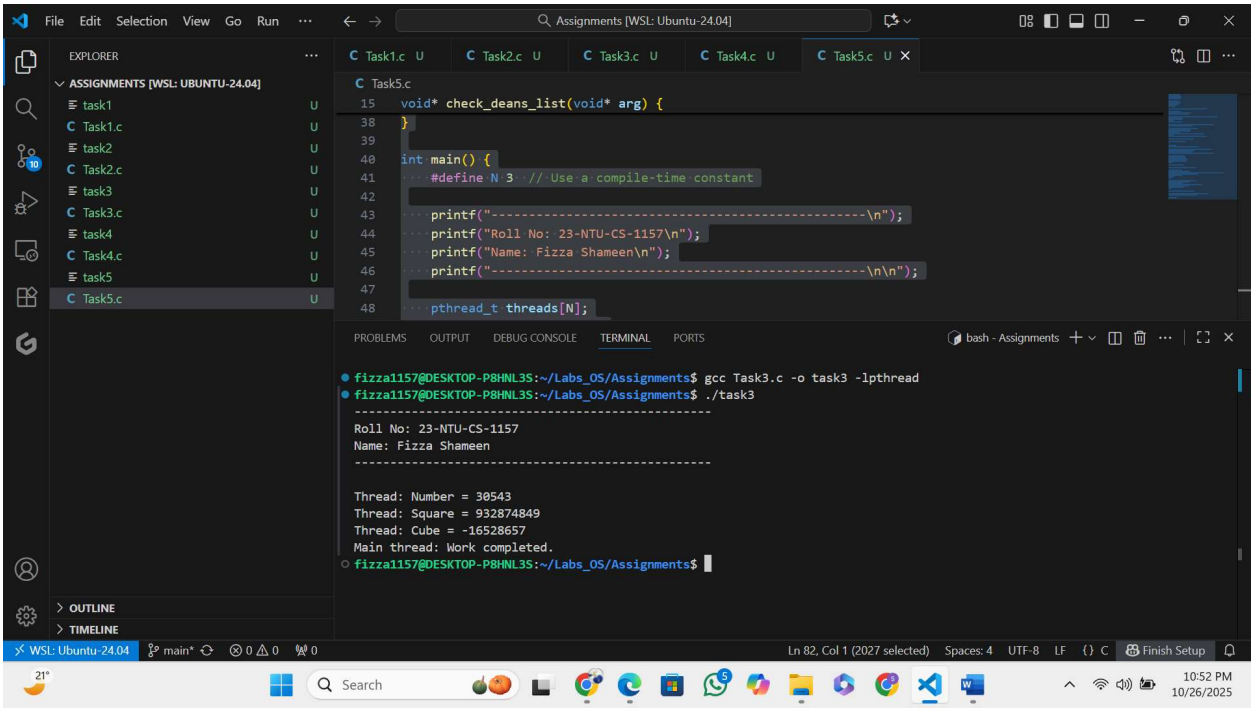
Solution:

Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>


//Function executed by each thread

void* calculate(void* arg){

  int num = *(int*)arg;   //Get the integer passed by main

  printf("Thread: Number = %d\n",num);

  printf("Thread: Square = %d\n",num * num);
```

```
    printf("Thread: Cube = %d\n",num * num * num);
    pthread_exit(NULL);
}
int main(){
    pthread_t thread;
    int num;
    printf("-----------------------------------------------\n");
    printf("Roll No: 23-NTU-CS-1157\n");
    printf("Name: Fizza Shameen\n");
    printf("-----------------------------------------------\n\n");

    //Create thread and pass the number
    if(pthread_create(&thread,NULL,calculate,&num)!=0){
        perror("Failed to create thread");
        return 1;
    }
    //Wait for thread to finish
    pthread_join(thread,NULL);
    printf("Main thread: Work completed.\n");
    return 0;
}
```

Output:



Task4:

Solution:

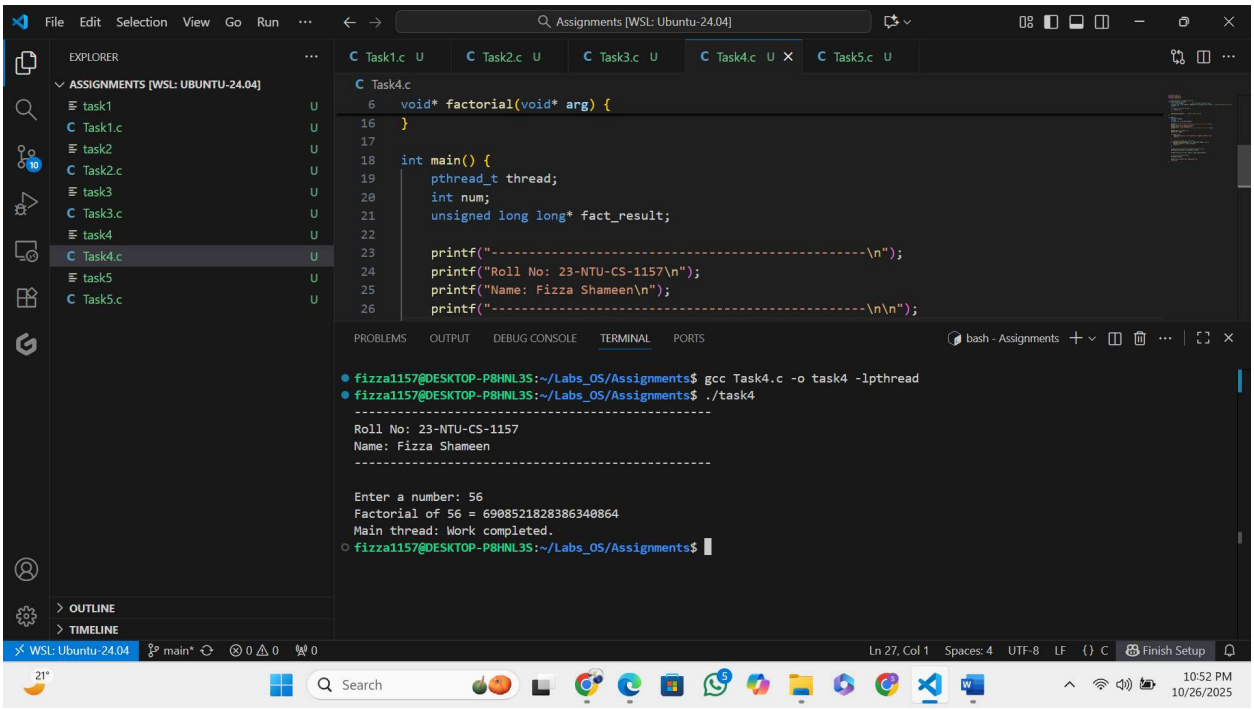
Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>


// Thread function to compute factorial

void* factorial(void* arg) {

  int n = *(int*)arg;       // Get the number passed by main

  unsigned long long* result = malloc(sizeof(unsigned long long)); // allocate memory for result

  *result = 1;
```

```
    for (int i = 1; i <= n; i++) {
        *result *= i;
    }

    pthread_exit(result);  // Return result pointer
}

int main() {
    pthread_t thread;
    int num;
    unsigned long long* fact_result;

    printf("-------------------------------------------------\n");
    printf("Roll No: 23-NTU-CS-1157\n");
    printf("Name: Fizza Shameen\n");
    printf("-------------------------------------------------\n\n");

    printf("Enter a number: ");
    scanf("%d", &num);

    if (num < 0) {
        printf("Factorial is not defined for negative numbers.\n");
        return 1;
    }

    // Create thread and pass the number
    if (pthread_create(&thread, NULL, factorial, &num) != 0) {
        perror("Failed to create thread");
```

```
    return 1;
  }

  // Wait for the thread to finish and receive result
  pthread_join(thread, (void**)&fact_result);

  printf("Factorial of %d = %llu\n", num, *fact_result);

  // Free the allocated memory
  free(fact_result);

  printf("Main thread: Work completed.\n");
  return 0;
}
```

Output:



Task5:

Solution:

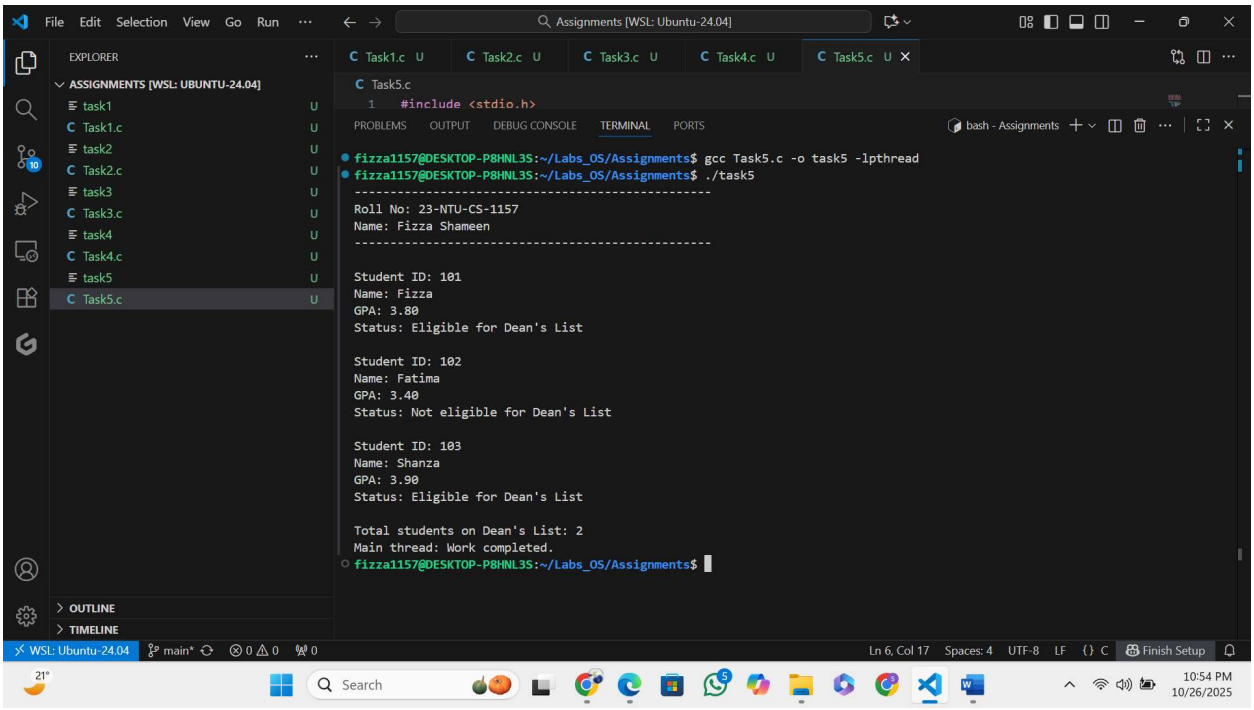
Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <string.h>

typedef struct {
    int student_id;
    char name[50];
    float gpa;
} Student;

pthread_mutex_t print_mutex; // mutex to serialize printing

// Thread function
void* check_deans_list(void* arg) {
    Student* s = (Student*)arg;

    int* eligible = malloc(sizeof(int)); // allocate memory to return result
    if (s->gpa >= 3.5)
        *eligible = 1;
    else
        *eligible = 0;
```

```
    // Lock mutex so this thread prints its block without interleaving
    pthread_mutex_lock(&print_mutex);

    printf("Student ID: %d\n", s->student_id);
    printf("Name: %s\n", s->name);
    printf("GPA: %.2f\n", s->gpa);
    if (*eligible)
        printf("Status: Eligible for Dean's List\n\n");
    else
        printf("Status: Not eligible for Dean's List\n\n");

    pthread_mutex_unlock(&print_mutex);

    pthread_exit(eligible);
}

int main() {
    #define N 3  // Use a compile-time constant

    printf("-------------------------------------------------\n");
    printf("Roll No: 23-NTU-CS-1157\n");
    printf("Name: Fizza Shameen\n");
    printf("-------------------------------------------------\n\n");

    pthread_t threads[N];
    Student students[N] = {
        {101, "Fizza", 3.8},
        {102, "Fatima", 3.4},
        {103, "Shanza", 3.9}
```

```
    };

    int dean_count = 0;

    pthread_mutex_init(&print_mutex, NULL);

    // Create threads
    for (int i = 0; i < N; i++) {
        if (pthread_create(&threads[i], NULL, check_deans_list, &students[i]) != 0) {
            perror("Failed to create thread");
            return 1;
        }
    }

    // Join threads and sum eligible counts
    for (int i = 0; i < N; i++) {
        int* eligible;
        pthread_join(threads[i], (void**)&eligible);
        dean_count += *eligible;
        free(eligible);
    }

    pthread_mutex_destroy(&print_mutex);

    printf("Total students on Dean's List: %d\n", dean_count);
    printf("Main thread: Work completed.\n");

    return 0;
}
```

Output:



Section B: Short Questions

2. Define an Operating System in a single line.

Answer:

An operating system is system software that manages computer hardware and software resources and provides services for computer programs.

3. What is the primary function of the CPU scheduler?

Answer:

The CPU scheduler selects one process from the ready queue to execute next on the CPU.

4. List any three states of a process.

Answer:

Three states of a process:

- Ready
- Running
- Waiting (Blocked).

5. What is meant by a Process Control Block (PCB)?

Answer:

A Process Control Block (PCB) is a data structure that stores all information about a process, such as process ID, state, registers, and scheduling info.

6. Differentiate between a process and a program.

Answer:

A program is a passive set of instructions; a process is an active instance of a program in execution.

7. What do you understand by context switching?

Answer:

Context switching is the process of saving the state of a running process and loading the state of another so the CPU can switch execution between them.

8. Define CPU utilization and throughput. 9. What is the turnaround time of a process?

Answer:

- **CPU utilization:** Percentage of time the CPU is actively executing processes.
- **Throughput:** Number of processes completed per unit time.

9. What is the turnaround time of a process?

Answer:

Turnaround time = Time from process submission to its completion.

10. How is waiting time calculated in process scheduling?

Answer:

Waiting time = Turnaround time − Burst time.

11. Define response time in CPU scheduling.

Answer:

Response time is the time from process submission until the first response is produced.

12. What is preemptive scheduling?

Answer:

Preemptive scheduling allows a process to be interrupted and moved to the ready queue before finishing.

13. What is non-preemptive scheduling?

Answer:

Non-preemptive scheduling allows a process to run until it completes or voluntarily releases the CPU.

14. State any two advantages of the Round Robin scheduling algorithm.

Answer:

Advantages of Round Robin (RR):

1. Fair time sharing among processes.
2. Suitable for time-sharing systems.

15. Mention one major drawback of the Shortest Job First (SJF) algorithm.

Answer:

Drawback of SJF: May cause starvation of longer processes.

16. Define CPU idle time.

Answer:

CPU idle time is the time during which the CPU remains unused (no process to execute).

17. State two common goals of CPU scheduling algorithms.

Answer:

Goals of CPU scheduling:

1. Maximize CPU utilization.
2. Minimize waiting and response time.

18. List two possible reasons for process termination.

Answer:

Reasons for process termination:

1. Normal completion.
2. Runtime error (e.g., divide by zero).

19. Explain the purpose of the wait() and exit() system calls.

Answer:

- **wait():** Used by a parent to wait for a child process to finish.
- **exit():** Used by a process to terminate its execution.

20. Differentiate between shared memory and message-passing models of inter-process communication.

Answer:

- **Shared memory:** Processes communicate by accessing a common memory region.
- **Message passing:** Processes exchange data using send/receive messages.

21. Differentiate between a thread and a process.

Answer:

- **Thread:** A lightweight subprocess sharing memory with others.
- **Process:** Independent execution unit with separate memory.

22. Define multithreading.

Answer:

Multithreading is the execution of multiple threads within a single process

23. Explain the difference between a CPU-bound process and an I/O-bound process.

Answer:

- **CPU-bound process:** Spends more time doing computations.
- **I/O-bound process:** Spends more time on input/output operations.

24. What are the main responsibilities of the dispatcher?

Answer:

The dispatcher gives control of the CPU to the process selected by the scheduler by performing context switching and switching to user mode.

25. Define starvation and aging in process scheduling.

Answer:

- **Starvation:** A process waits indefinitely due to resource unavailability.
- **Aging:** Gradual increase in a process's priority to prevent starvation.

26. What is a time quantum (or time slice)?

Answer:

A time quantum (time slice) is the fixed amount of CPU time given to each process in Round Robin scheduling.

27. What happens when the time quantum is too large or too small?

Answer:

- Too large → behaves like FCFS (poor response time).
- Too small → too many context switches (overhead).

28. Define the turnaround ratio (TR/TS).

Answer:

Turnaround ratio (TR/TS) = Actual turnaround time / Service time.

29. What is the purpose of a ready queue?

Answer:

The ready queue holds all processes waiting to use the CPU.

30. Differentiate between a CPU burst and an I/O burst.

Answer:

- **CPU burst:** Time the process spends executing on CPU.
- **I/O burst:** Time the process waits for I/O operations.

31. Which scheduling algorithm is starvation-free, and why?

Answer:

Round Robin (RR) is starvation-free because every process gets CPU time in a cyclic order.

32. Outline the main steps involved in process creation in UNIX.

Answer:

Process creation in UNIX:

1. Parent process calls fork() to create a child.
2. Child gets a copy of parent's address space.
3. Child may call exec() to run a new program.

33. Define zombie and orphan processes.

Answer:

- **Zombie process:** Finished execution but still has an entry in the process table.

- **Orphan process:** Parent terminated before the child process.

34. Differentiate between Priority Scheduling and Shortest Job First (SJF).

Answer:

- **Priority Scheduling:** Based on process priority.
- **SJF:** Based on shortest CPU burst time.

35. Define context switch time and explain why it is considered overhead.

Answer:

Context switch time is the time required to save and load process states; it's overhead because no useful work is done during it.

36. List and briefly describe the three levels of schedulers in an Operating System.

Answer:

Three levels of schedulers:

1. **Long-term (Job Scheduler):** Admits jobs into the system.
2. **Short-term (CPU Scheduler):** Chooses process for CPU.
3. **Medium-term (Swapper):** Manages suspended processes.

37. Differentiate between User Mode and Kernel Mode in an Operating System.

Answer:

- **User Mode:** Limited access; used for user applications.
- **Kernel Mode:** Full access to hardware; used for OS operations.

Section C: Technical / Analytical Questions

1. Describe the complete life cycle of a process with a neat diagram showing transitions between New, Ready, Running, Waiting, and Terminated states.

Answer:

Process Life Cycle:

A process passes through several states during its execution.
The main states are New, Ready, Running, Waiting, and Terminated.

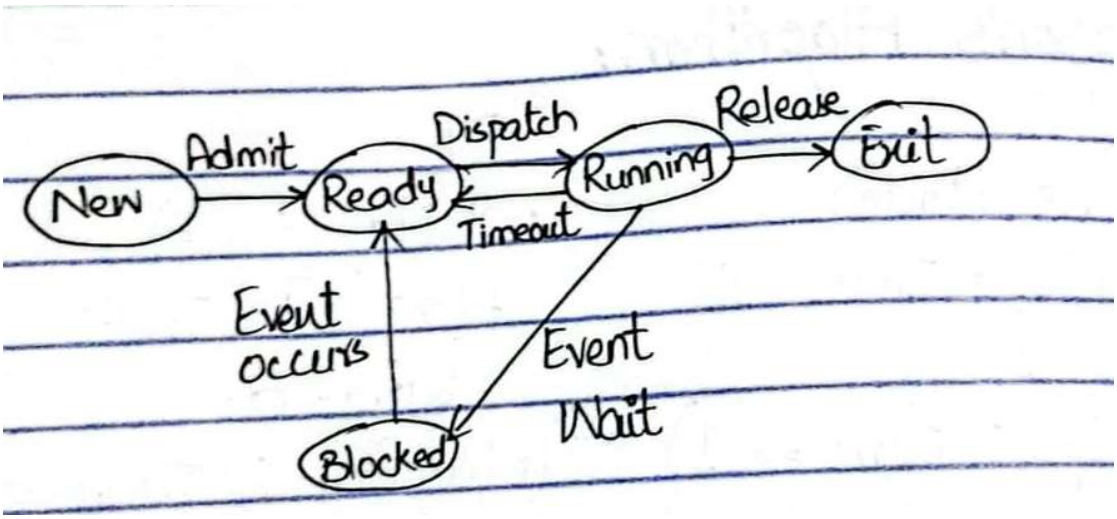
Description:

- New: Process is being created.
- Ready: Process is waiting to be assigned to the CPU.
- Running: Process is currently being executed on the CPU.
- Waiting (Blocked): Process is waiting for some I/O operation or event to complete.
- Terminated: Process has finished execution.

State Transitions:

- New → Ready: Process admitted by the long-term scheduler.
- Ready → Running: CPU scheduler dispatches the process.
- Running → Waiting: Process requests I/O.
- Waiting → Ready: I/O completed; process returns to ready queue.
- Running → Terminated: Process finishes execution.
- Running → Ready: Preemption by scheduler (in preemptive systems).

Diagram:



2. Write a short note on context switch overhead and describe what information must be saved and restored.

Answer:

Context Switch Overhead:

A context switch occurs when the CPU switches from one process (or thread) to another.

Overhead Explanation:

- During a context switch, the CPU performs no useful work—it only saves and restores states.
- Hence, context switching time is considered overhead since it consumes CPU time without advancing any process execution.

Information Saved and Restored:

When switching:

- **Saved (old process):**
    - CPU registers
    - Program counter (PC)
    - Stack pointer
    - Process state
    - Memory management info (page tables, etc.)
    - I/O status and accounting info
- **Restored (new process):**

    All of the above for the next process (from its PCB)

Goal: Minimize context switching overhead to improve CPU efficiency.

3. List and explain the components of a Process Control Block (PCB).

Answer:

Components of Process Control Block (PCB):

A Process Control Block (PCB) is a data structure maintained by the OS to store information about a specific process.

Main Components:

1. **Process Identification Data**
    - Process ID (PID)
    - Parent process ID
    - User ID
2. **Process State**
    - Current state: New, Ready, Running, Waiting, Terminated
3. **CPU Registers**

- General-purpose registers, program counter, stack pointer

4. **CPU Scheduling Information**
    - Priority, scheduling queue pointers, CPU burst info
5. **Memory Management Information**
    - Base and limit registers, page or segment tables
6. **Accounting Information**
    - CPU used, execution time, job or process number
7. **I/O Status Information**
    - List of open files, allocated I/O devices, pending I/O requests

4. Differentiate between Long-Term, Medium-Term, and Short-Term Schedulers with examples.

Answer:

Types of Schedulers

Scheduler Type	Function	Frequency	Example / Role
Long-Term (Job Scheduler)	Selects which jobs/processes are admitted into the system (into ready queue).	Runs infrequently	Controls degree of multiprogramming. Example: Decides which jobs enter memory.
Medium-Term Scheduler	Temporarily removes or suspends processes from main memory (swapping).	Occasionally	Improves performance by freeing memory for other processes.
Short-Term (CPU Scheduler)	Selects which ready process gets the CPU next.	Very frequent (milliseconds)	Dispatches processes for execution. Example: FCFS, SJF, RR, Priority.

5. Explain CPU Scheduling Criteria (Utilization, Throughput, Turnaround, Waiting, and Response) and their optimization goals.

Answer:

CPU Scheduling Criteria and Optimization Goals

Criterion	Definition	Optimization Goal
CPU Utilization	Percentage of time the CPU is actively working.	Maximize
Throughput	Number of processes completed per unit time.	Maximize
Turnaround Time	Time taken from process submission to completion.	Minimize
Waiting Time	Total time a process spends waiting in the ready queue.	Minimize
Response Time	Time from request submission until first response is produced.	Minimize

Section D: CPU Scheduling Calculations

Perform the following calculations for each part (A–C).

a) Draw Gantt charts for FCFS, RR (Q=4), SJF, and SRTF.

b) Compute Waiting Time, Turnaround Time, TR/TS ratio, and CPU Idle Time.

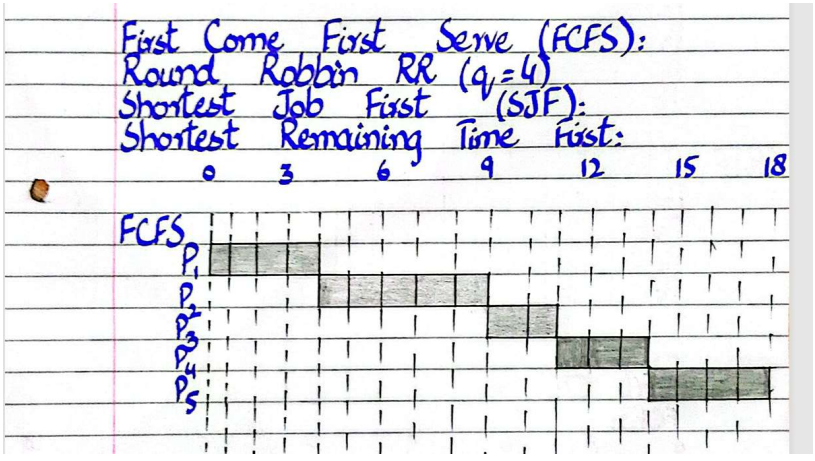
c) Compare average values and identify which algorithm performs best.

Part-A :

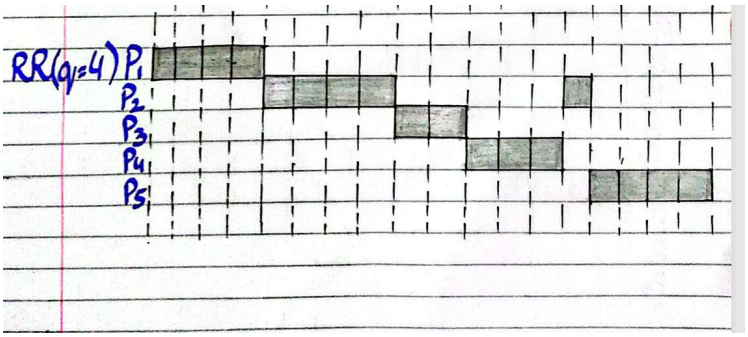
a) Draw Gantt charts for FCFS, RR (Q=4), SJF, and SRTF.

Gantt Chart:

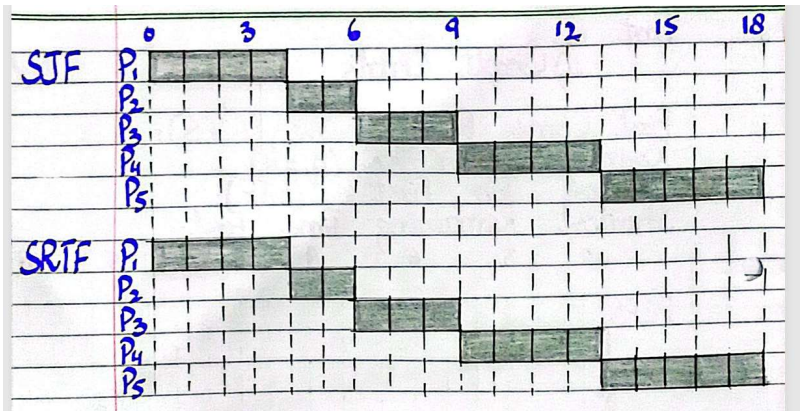
FCFS:



RR(q=4):



SJF and SRTF:



b) Compute Waiting Time, Turnaround Time, TR/TS ratio, and CPU Idle Time.

FCFS:

Process	Arrival Time	Service Time	Completion Time	Waiting Time	TurnAround Time(TAT)	TR/TS	CPU Idle Time
P1	0	4	4	0	4	1.00	0
P2	2	5	9	2	7	1.40	0
P3	4	2	11	5	7	3.50	0
P4	6	3	14	5	8	2.67	0
P5	9	4	18	5	9	2.25	0

- Average waiting time = (0 + 2 + 5 + 5 + 5) / 5 = 3.4
- Average turnaround time = (4 + 7 + 7 + 8 + 9) / 5 = 7.0
- CPU idle time = 0 (first arrival at t=0)

RR(q=4):

Process	Arrival Time	Service Time	Completion Time	Waiting Time	TurnAround Time(TAT)	TR/TS	CPU Idle Time

P1	0	4	4	0	4	1.00	0
P2	2	5	14	7	12	2.40	0
P3	4	2	10	4	6	3.00	0
P4	6	3	13	4	7	2.33	0
P5	9	4	18	5	9	2.25	0

- Average waiting time = (0 + 7 + 4 + 4 + 5) / 5 = 4.0
- Average turnaround time = (4 + 12 + 6 + 7 + 9) / 5 = 7.6
- CPU idle time = 0

SJF:

Process	Arrival Time	Service Time	Completion Time	Waiting Time	TurnAround Time(TAT)	TR/TS	CPU Idle Time
P1	0	4	4	0	4	1.00	0
P2	2	5	18	11	16	3.20	0
P3	4	2	6	0	2	1.00	0
P4	6	3	9	0	3	1.00	0
P5	9	4	13	0	4	1.00	0

- Avg waiting = (0+0+4+0+5)/5 = 1.8
- Avg turnaround = (3+2+9+4+11)/5 = 5.8
- CPU idle time = 0

SRTF:

Process	Arrival Time	Service Time	Completion Time	Waiting Time	TurnAround Time(TAT)	TR/TS	CPU Idle Time
P1	0	4	4	0	4	1.00	0
P2	2	5	18	11	16	3.20	0
P3	4	2	6	0	2	1.00	0
P4	6	3	9	0	3	1.00	0
P5	9	4	13	0	4	1.00	0

- Average waiting time = (0 + 0 + 0 + 0 + 11) / 5 = 2.2
- Average turnaround time = (4 + 2 + 3 + 4 + 16) / 5 = 5.8
- CPU idle time = 0

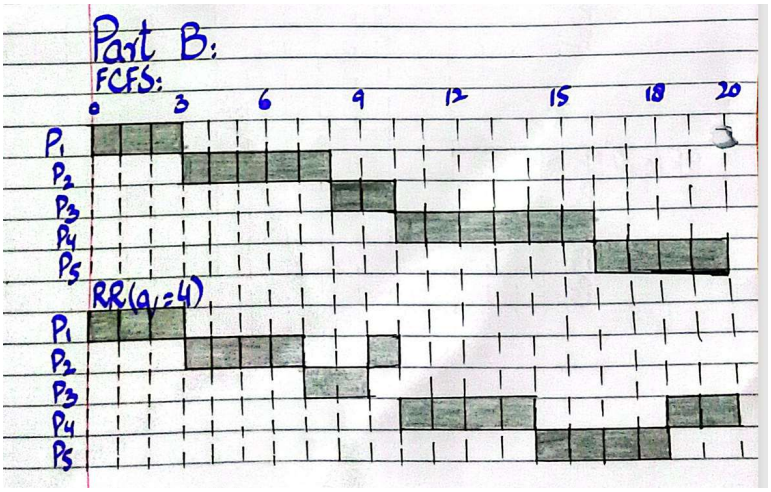
c) Compare average values and identify which algorithm performs best.

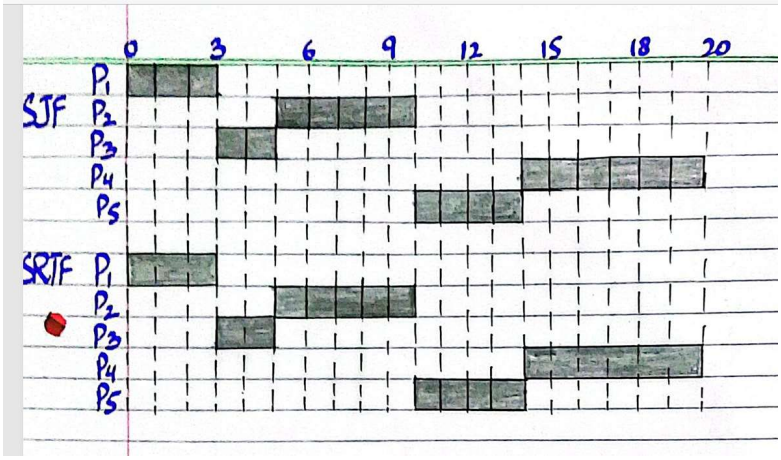
Algorithm	Average Waiting Time	Average Turnaround Time
FCFS	3.4	7.0
RR(q=4)	4.0	7.6
SJF	2.2	5.8
SRTF	2.2	5.8

Part-B:

a) Draw Gantt charts for FCFS, RR (Q=4), SJF, and SRTF.

Gantt Chart:





b) Compute Waiting Time, Turnaround Time, TR/TS ratio, and CPU Idle Time.

FCFS:

Process	Arrival Time	Service Time	Completion Time	Waiting Time	TurnAround Time(TAT)	TR/TS	CPU Idle Time
P1	0	4	3	0	3	1.00	0
P2	2	5	8	2	7	1.40	0
P3	4	2	10	5	7	3.50	0
P4	6	3	16	1	7	1.17	0
P5	9	4	20	6	10	2.50	0

- Avg waiting = (0+2+5+1+6)/5 = 2.8
- Avg turnaround = (3+7+7+7+10)/5 = 6.8

- CPU idle time = 0 (first arrival at t=0; no gaps)

RR(q=4):

Process	Arrival Time	Service Time	Completion Time	Waiting Time	TurnAround Time(TAT)	TR/TS	CPU Idle Time
P1	0	4	3	0	3	1.00	0
P2	2	5	10	4	9	1.80	0
P3	4	2	9	4	6	3.00	0
P4	6	3	20	5	11	1.83	0
P5	9	4	18	4	8	2.00	0

- Avg waiting = (0+4+4+5+4)/5 = 3.4
- Avg turnaround = (3+9+6+11+8)/5 = 7.4
- CPU idle time = 0

SJF:

Process	Arrival Time	Service Time	Completion Time	Waiting Time	TurnAround Time(TAT)	TR/TS	CPU Idle Time
P1	0	4	3	0	3	1.00	0
P2	2	5	10	4	9	1.80	0
P3	4	2	5	0	2	1.80	0
P4	6	3	20	5	11	1.83	0
P5	9	4	14	0	4	1.80	0

- Avg waiting = (0+0+4+0+5)/5 = 1.8
- Avg turnaround = (3+2+9+4+11)/5 = 5.8
- CPU idle time = 0

SRTF:

Process	Arrival Time	Service Time	Completion Time	Waiting Time	TurnAround Time(TAT)	TR/TS	CPU Idle Time
P1	0	4	3	0	3	1.00	0
P2	2	5	10	4	9	1.80	0
P3	4	2	5	0	2	1.80	0
P4	6	3	20	5	11	1.83	0
P5	9	4	14	0	4	1.80	0

- Avg waiting = 1.8
- Avg turnaround = 5.8
- CPU idle time = 0

c) Compare average values and identify which algorithm performs best.

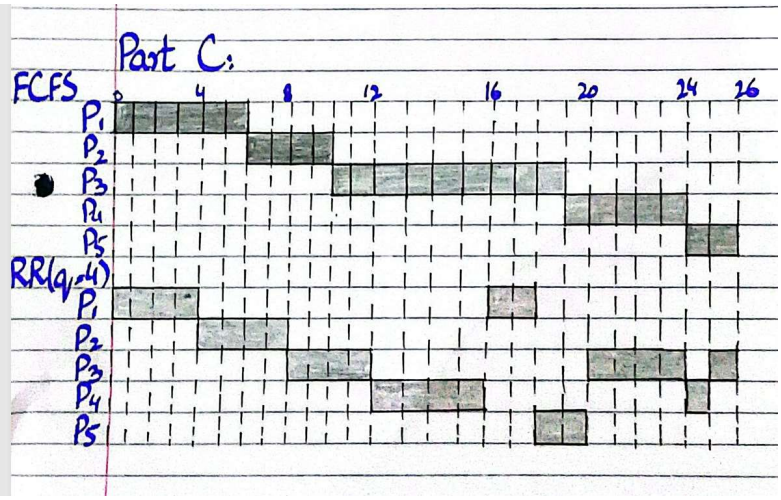
Algorithm	Average Waiting Time	Average Turnaround Time

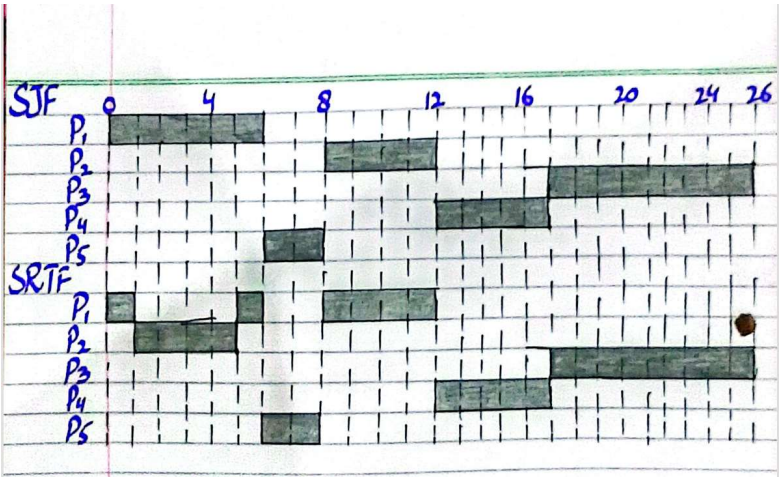
FCFS	2.8	6.8
RR(q=4)	3.4	7.4
SJF	1.8	5.8
SRTF	1.8	5.8

Part-C:

a) Draw Gantt charts for FCFS, RR (Q=4), SJF, and SRTF.

Gantt Chart:





b) Compute Waiting Time, Turnaround Time, TR/TS ratio, and CPU Idle Time.

FCFS:

Process	Arrival Time	Service Time	Completion Time	Waiting Time	TurnAround Time(TAT)	TR/TS	CPU Idle Time

P1	0	4	6	0	6	1.00	0
P2	2	5	10	5	9	2.25	0
P3	4	2	19	8	17	1.89	0
P4	6	3	24	16	21	4.20	0
P5	9	4	26	18	20	10.00	0

- Avg waiting time = (0 + 5 + 8 + 16 + 18)/5 = 9.4
- Avg turnaround time = (6 + 9 + 17 + 21 + 20)/5 = 14.6
- CPU idle time = 0

RR(q=4):

Process	**Arrival Time**	**Service Time**	**Completion Time**	**Waiting Time**	**TurnAround Time(TAT)**	**TR/TS**	**CPU Idle Time**
P1	0	4	18	12	18	3.00	0
P2	2	5	8	3	7	1.75	0
P3	4	2	26	15	24	2.67	0
P4	6	3	25	17	22	4.40	0
P5	9	4	20	12	14	7.00	0

- Avg waiting time = (12+3+15+17+12)/5 = 11.8
- Avg turnaround time = (18+7+24+22+14)/5 = 17.0
- CPU idle time = 0

SJF:

Process	**Arrival Time**	**Service Time**	**Completion Time**	**Waiting Time**	**TurnAround Time(TAT)**	**TR/TS**	**CPU Idle Time**
P1	0	4	6	0	6	1.00	0
P2	2	5	12	0	2	1.00	0
P3	4	2	26	7	11	2.75	0
P4	6	3	17	9	14	2.80	0
P5	9	4	8	15	24	2.67	0

- Avg waiting time = (0+0+7+9+15)/5 = 6.2
- Avg turnaround time = (6+2+11+14+24)/5 = 11.4
- CPU idle time = 0

SRTF:

Process	**Arrival Time**	**Service Time**	**Completion Time**	**Waiting Time**	**TurnAround Time(TAT)**	**TR/TS**	**CPU Idle Time**
P1	0	4	12	6	12	2.00	0
P2	2	5	5	0	4	1.00	0
P3	4	2	26	15	24	2.67	0
P4	6	3	17	9	14	2.80	0
P5	9	4	8	0	2	1.00	0

- Avg waiting time = (6+0+15+9+0)/5 = 6.0

- Avg turnaround time = (12+4+24+14+2)/5 = 11.2
- CPU idle time = 0

c) Compare average values and identify which algorithm performs best.

Algorithm	Average Waiting Time	Average Turnaround Time
FCFS	9.4	14.6
RR(q=4)	11.8	17.0
SJF	6.2	11.4
SRTF	6.0	11.2